

Miguel Areias

FCUP - Department of Computer Science

Adapted from the slides of

Luís Lopes, Ricardo Rocha and Sérgio Crisóstomo

Compilation

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello world!\n");  
    return 0;  
}
```

Main compilation flags:

```
$ gcc -o hello hello.c // -o: specify executable name  
$ gcc -Wall hello.c    // -Wall: print all warnings  
$ gcc -g hello.c       // -g: enable debugging (e.g., gdb)  
$ gcc -O hello.c       // -O: turn on optimization (don't)
```

Data Types: Examples

```
char a = 'A';
```

```
char b = 65;
```

```
char c = 0x41;
```

```
int i = -2343234;
```

```
unsigned int ui = 4294967295;
```

```
unsigned int uj = ((long) 1 << 32) - 1;
```

```
unsigned int uk = -1;
```

```
float pi = 3.14;
```

```
double long_pi = 0.31415e+1;
```

Integer Data Types

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

- There is no explicit boolean data type:
 - Values **0** or **NULL** mean **FALSE**
 - Any other value means **TRUE**

Floating-Point Data Types

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

- To get the exact size of a data type or variable, use the `sizeof()` operator that yields the **storage size of the type or variable in bytes**:

```
int x;
```

```
printf("storage size for type int: %d bytes\n", sizeof(int));
```

```
printf("storage size for variable x: %d bytes\n", sizeof(x));
```

Void Data Type

- The **void** data type specifies that **no value is available or that its type is undefined** and it can be used in three scenarios:
 - Functions returning no values: **void exit(int)**
 - Functions taking no arguments: **int rand(void)**
 - Pointer to undefined data type: **void *malloc(size_t)**

Arithmetic Operators

Operator	Description
+	Adds two operands
-	Subtracts second operand from the first
*	Multiplies both operands
/	Divides numerator by de-numerator
%	Modulus Operator and remainder of after an integer division
++	Increments operator increases integer value by one
--	Decrements operator decreases integer value by one

Relational Operators

Operator	Description
<code>==</code>	Checks if the values of two operands are equal or not, if yes then condition becomes true.
<code>!=</code>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
<code>></code>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.
<code><</code>	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.
<code>>=</code>	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.
<code><=</code>	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.

Logical Operators

Operator	Description
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.
	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.

Bitwise Operators

Operator	Description
&	Binary AND Operator copies a bit to the result if it exists in both operands.
	Binary OR Operator copies a bit if it exists in either operand.
^	Binary XOR Operator copies the bit if it is set in one operand but not both.
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.

Decision Making Statements

□ If statement

```
if (cond) {  
    ... // continue here if cond is true  
else {  
    ... // continue here if cond is false  
}
```

□ Switch statement

```
switch (val) {  
    case const1: ... // continue here if val is const1  
        break;  
    ...  
    default: ... // continue here if all other cases fail  
}
```

Loop Control Statements

□ For statement

```
for (pre_cond; cond; post_loop) {  
    ... // execute pre_cond once and loop while cond is true  
}
```

□ while statement

```
while (cond) {  
    ... // loop while cond is true  
}
```

□ Do/while statement

```
do {  
    ... // execute once and loop while cond is true  
} while (cond);
```

Pointers

- A **pointer** is a variable whose value is a memory address
- The general form of a pointer variable declaration is

type* varname;

where **type** is the pointer's base data type and **varname** is the name of the pointer variable:

char* p1; // pointer to a character

int* p2; // pointer to an integer

float* p3; // pointer to a float

- There are two main pointer operations:
 - **&** to obtain the memory address of a variable
 - ***** to access the contents of a memory address stored in a pointer

Pointers: Example I

```
int main () {  
    int v = 0;  
    int *ip; // initially ip stores an undefined memory address  
    ip = &v; // here ip stores the memory address of variable v  
    // print address stored in pointer ip  
    printf("value stored in ip variable: %p\n", ip);  
    // change v's value using pointer ip  
    *ip = 10;  
    // print v's value using pointer ip  
    printf("value stored in v variable: %d\n", *ip);  
    return 0;  
}
```

Example

```
int main() {  
    int i, j, *p, *q;  
    i = 5;  
    p = &i;  
    *p = 7;  
    j = 3;  
    p = &j;  
    q = p;  
    *q = 2;  
    *p = 9;  
    return 0;  
}
```

Pointers: Example II

```
int i, j, *ip, *ip2;
```

```
// step 1
```

```
i = 5;
```

```
ip = &i;
```

```
// step 2
```

```
*ip = 7;
```

```
j = 3;
```

```
ip = &j;
```

```
// step 3
```

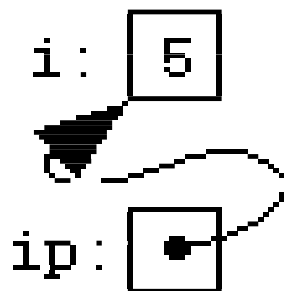
```
ip2 = ip;
```

```
// step 4
```

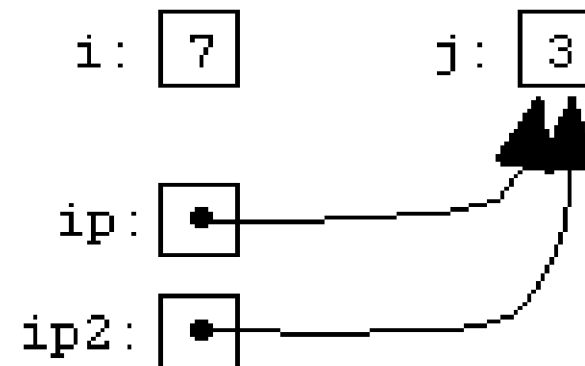
```
ip = &i;
```

```
...
```

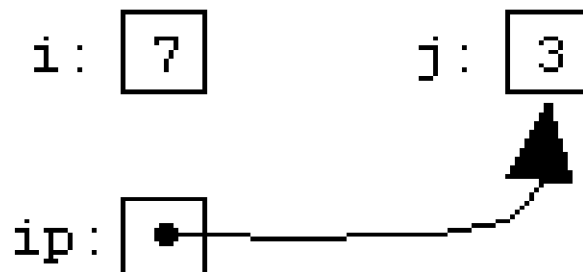
step 1



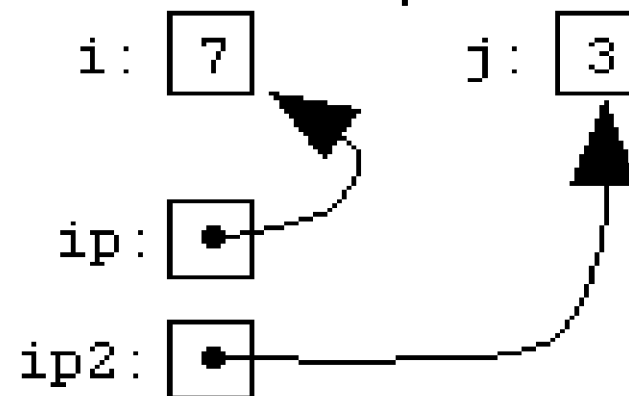
step 3



step 2



step 4



Arrays

- Arrays consist of contiguous memory locations used to store a **fixed-size sequential collection of elements of the same type**
- Each specific element in an array is **accessed by an index**
 - The lowest index (0) corresponds to the first element and the highest address to the last element
- The general form of an array variable declaration is

type arrayName[size];

where **type** is the data type of the elements, **arrayName** is the name of the array variable and **size** is the number of elements in the array:

```
char cs[10]; // an array of 10 characters
```

```
int is[5]; // an array of 5 integers
```

Arrays

- Arrays can also be initialized when declaring them:

```
int is[5] = {1000, 2, 3, 7, 50};
```

```
int is[] = {1000, 2, 3, 7, 50};
```

- When declaring the size of the array, the number of values between braces { } cannot be larger than the number of elements declared for the array between square brackets []
- When omitting the size of the array, an array just big enough to hold the initialization is created

Arrays: Example

```
int main () {  
    int i;  
    // declare an array of 10 integers  
    int n[10];  
    // initialize elements of array  
    for (i = 0; i < 10; i++)  
        n[i] = i + 100;  
    // output each array element's value  
    for (i = 0; i < 10; i++)  
        printf("element[%d] = %d\n", i, n[i]);  
    return 0;  
}
```

Strings

- A string is an **array of characters terminated by the null character '\0'**
- To hold the null character, the size of the array is one more than the number of characters in the string:

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

- Following the array initialization rule, we can also write:

```
char greeting[] = "Hello";
```

```
char *greeting = "Hello";
```

Basic String Functions

Function & Purpose

strcpy(s1, s2);

Copies string s2 into string s1.

strcat(s1, s2);

Concatenates string s2 onto the end of string s1.

strlen(s1);

Returns the length of string s1.

strcmp(s1, s2);

Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.

strchr(s1, ch);

Returns a pointer to the first occurrence of character ch in string s1.

strstr(s1, s2);

Returns a pointer to the first occurrence of string s2 in string s1.

Basic String Functions: Example

```
int main () {  
    char str1[100] = "Hello";  
    char str2[100] = "World";  
    // concatenate str2 onto the end of str1  
    strcat(str1, str2);  
    // copy str1 into str2  
    strcpy(str2, str1);  
    // print length of str1  
    printf("str1 length: %d\n", strlen(str1));  
    return 0;  
}
```

Functions

- While calling a function, there are two ways that arguments can be passed to a function:
 - **Call by value** copies the actual value of an argument into the formal parameter of the function, meaning that changes made to the parameter inside the function have no effect on the argument
 - **Call by reference** copies the address of an argument into the formal parameter and, inside the function, the address is used to access the actual argument used in the call, meaning that changes made to the parameter affect the argument
- By default, **C uses call by value to pass arguments**
- However, if we use **pointer arguments** in a function, we can implement a kind of **call by reference for the variables pointed by the argument pointers** and thus alter those variables inside the function

Call by Value: Example

```
void swap_call_by_value(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int i = 1;  
    int j = 2;  
    swap_call_by_value(i, j);  
    printf("i: %d j: %d\n", i, j); // prints 'i:1 j:2'  
    return 0;  
}
```


Call by Reference: Example

```
void swap_call_by_reference(int *p, int *q) {  
    int temp = *p;  
    *p = *q;  
    *q = temp;  
}  
  
int main() {  
    int i = 1;  
    int j = 2;  
    swap_call_by_reference(&i, &j);  
    printf("i: %d j: %d\n", i, j); // prints 'i:2 j:1'  
    return 0;  
}
```

Dynamic Memory Allocation: Wrong!

```
#include <stdio.h>
```

```
int* get_int() { int i = 2; return &i; }
```

```
int use_int(int *p) { int j = 3; return j + *p; }
```

```
int main() {  
    int* p = get_int();  
    printf("integer = %d\n", *p);  
    int k = use_int(p);  
    printf("integer = %d\n", k);  
    return 0;  
}
```

Dynamic Memory Allocation: Right!

```
#include <stdio.h>
#include <stdlib.h>

int* get_int() {
    int* p = (int*)malloc(sizeof(int));
    *p = 2; return p; }

int use_int(int *p) { int j = 3; return j + *p; }

int main() {
    int* p = get_int(); printf("integer = %d\n", *p);
    int k = use_int(p); printf("integer = %d\n", k);
    return 0; }
```

Dynamic Memory Allocation

Function and Description

void *calloc(int num, int size);

This function allocates an array of **num** elements each of which size in bytes will be **size**.

void free(void *address);

This function release a block of memory block specified by address.

void *malloc(int num);

This function allocates an array of **num** bytes and leave them initialized.

void *realloc(void *address, int newsize);

This function re-allocates memory extending it upto **newsize**.

Dynamic Memory Allocation

```
int main() {  
    // make space for string at heap  
    char *name = (char *) malloc(20 * sizeof(char));  
    if (name == NULL) {  
        /* malloc error */  
    } else {  
        strcpy(name, "Hello world!");  
        printf("%s\n", name);  
    }  
    // free the memory space previously allocated  
    free(name);  
    return 0;  
}
```

Structures

- A structure is a user defined data type which allows to **combine data items of different kinds**
- The **struct** statement defines a new data type which can contain several different item members:

```
struct [name] {  
    type1 item1;  
    type2 item2;  
    ...  
    typeN itemN;  
} [variables];
```

Structures: Example

```
struct list_elem {  
    int data;  
    struct list_elem *next;  
};  
  
int main() {  
    struct list_elem le = {10, NULL};  
    struct list_elem* lep = &le;  
    printf("data %d next %p\n", le.data, le.next);  
    printf("data %d next %p\n", (*lep).data, (*lep).next);  
    printf("data %d next %p\n", lep->data, lep->next);  
    return 0;  
}
```

Typedefs

- The keyword **typedef** allows to **give a type a new identifier**:

```
typedef unsigned char byte;
```

- The new identifier can then be used as an abbreviation for the type:

```
byte b1, b2;
```

- The typedef keyword can also be used to give a name to user defined data types.

```
typedef struct { /* anonymous struct */  
    int data;  
    struct list_elem *next;  
} list_elem;  
...  
list_elem le = {10, NULL};
```


Command Line Arguments

```
int main(int argc, char* argv[]) {  
    if (argc == 2)  
        printf("Hello %s!\n", argv[1]);  
    else  
        printf("Hello world!\n");  
    return 0;  
}
```

Running example:

```
$ gcc -o prog prog.c
```

```
$ ./prog
```

```
Hello world!
```

```
$ ./prog sd0
```

```
Hello sd0!
```

The C Preprocessor

- The **C Preprocessor** is not part of the compiler, but a separate step in the compilation process that works like a text substitution tool

Directive	Description
<code>#define</code>	Substitutes a preprocessor macro
<code>#include</code>	Inserts a particular header from another file
<code>#undef</code>	Undefines a preprocessor macro
<code>#ifdef</code>	Returns true if this macro is defined
<code>#ifndef</code>	Returns true if this macro is not defined
<code>#if</code>	Tests if a compile time condition is true
<code>#else</code>	The alternative for <code>#if</code>
<code>#elif</code>	<code>#else</code> and <code>#if</code> in one statement
<code>#endif</code>	Ends preprocessor conditional
<code>#error</code>	Prints error message on stderr

The C Preprocessor

□ Examples:

- `#include <stdio.h>` `/* header for I/O library */`
- `#define PI ((double)3.141592653589793)` `/* math constant */`
- `#define MAX_VALUE 1000` `/* program constant */`
- `#define MAX(x,y) (((x)<(y))? (y) : (x))` `/* inlined functions */`
- `#ifdef DEBUG` `/* conditional compilation */`
`printf("This is a debugging message\n");`
`#endif`